

Programação Desenvolvimento Web II

Projeto Agenda



Se está lendo este material, é porquê está continuando sua jornada para trabalhar com o desenvolvimento para internet.

Esta OT (orientação técnica) propõe a criação de um sistema de contato (crud) usando as tecnologias vistas até aqui.

Essa é uma atividade avaliativa. Baseado no livro PHP Profissional.

O sistema deve ser entregue com tudo funcionando, valendo a seguinte pontuação de forma individual:

- 1 – Listar dados (2,0);
- 2 – Cadastrar dados (2,0);
- 3 – Alterar dados (2,0);
- 4 – Incluir dados (2,0);
- 5 – Aplicar os designs mostrados nas imagens (1,0);
- 6 – Adicionar o campo site varchar(100) no banco de dados e fazer aparecer na inclusão, edição e listagem de dados (1,0).

3.18 Um projeto simples de uma agenda: BootStrap, jQuery e PHP em ação

Para fechar esse capítulo vamos construir um projeto simples em PHP envolvendo os conceitos vistos até aqui. Esse projeto trata de uma pequena agenda de contatos, onde iremos realizar as operações de inclusão, alteração, exclusão e consulta aos dados. Esse conjunto de operações é conhecido também pela expressão CRUD em inglês.

Iremos utilizar também algumas bibliotecas amplamente usadas no desenvolvimento web, e que não foram abordadas no capítulo. Mas que são relevantes em se pensando desenvolver aplicações PHP atualmente, principalmente levando em conta o lado cliente. Essas tecnologias são respectivamente Bootstrap (CSS) e jQuery (JavaScript). Será descrito os pontos principais dessas bibliotecas ao longo do projeto, e complementado as explicações com uma visão geral delas nos apêndices do livro.

3.18.1 Criando a tabela contato

Na seção sobre conexão com banco de dados criamos uma base de dados chamada de `agenda`. Vamos agora criar a tabela que iremos popular com informações, e que ficará nessa base.

Para tanto vamos usar novamente a ferramenta de linha de comando do MySQL, acessível em MySQL → MySQL 5.7 Command Line Client. Informe as informações para login demandadas pela ferramenta e não esqueça de selecionar o banco `agenda` usando o comando SQL `USE agenda;`.

 contato.sql

```
CREATE TABLE `contato` (  
  `id` int(11) NOT NULL AUTO_INCREMENT,  
  `nome` varchar(50) NOT NULL,  
  `telefone` varchar(50) NULL,  
  `email` varchar(50) NULL,  
  `endereco` TEXT NULL,  
  PRIMARY KEY (`id`)  
);
```

 Resultado:

Query OK, 0 rows affected (0.03 sec)

Com a tabela devidamente criada podemos partir para o próximo passo que é a criação do projeto no Eclipse.

3.18.2 Criando o projeto agenda no Eclipse

Para criar um projeto no Eclipse com a IDE aberta vá em File → New → PHP Project. A figura 3.25 mostra as informações do projeto, depois das informações preenchidas clique em *Finish*:

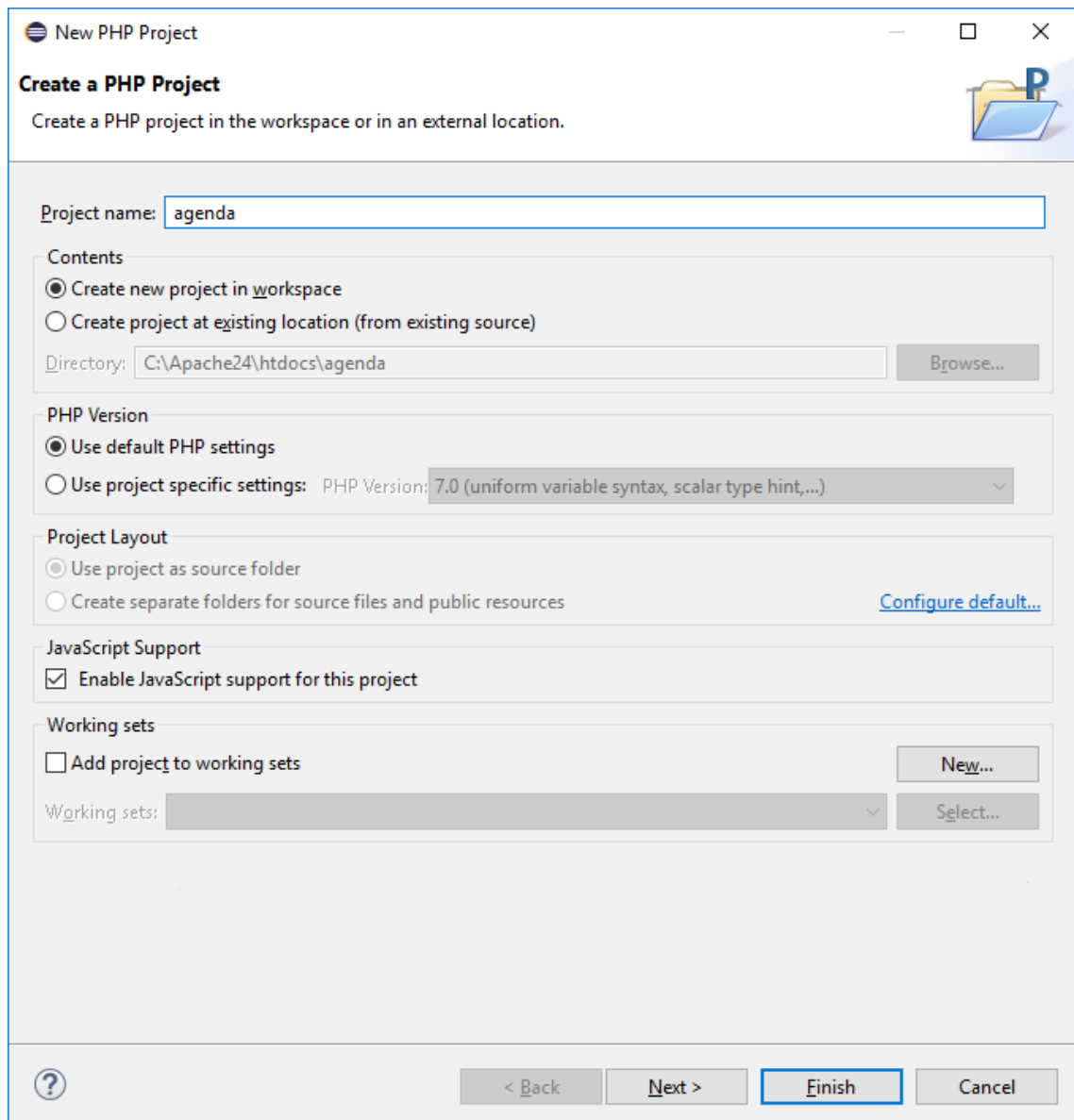


Figura 3.25 —Projeto sendo criado.

Com o projeto criado vamos adicionar algumas pastas ao mesmo. Essas pastas seguem um padrão de separação para arquivos acessórios ao PHP como JavaScript e estilos CSS. Além dos arquivos das bibliotecas que vamos usar.

Inicialmente vamos criar a pasta `resources` (recursos). Para fazer isso, clique com o botão direito no nome do projeto e acesse a opção `New → Folder`. Com a pasta `resources` criada, clique com o botão direito sobre ela, vamos criar mais duas pastas dentro dela, denominadas `common` e `lib` novamente via opção `New → Folder`. A primeira pasta é aonde vamos guardar os arquivos acessórios da aplicação e que foram construídos por nós. A segunda pasta irá guardar os arquivos acessórios de terceiros e que também são utilizados na aplicação. Essa separação é útil para entendermos o que foi construído por nós e o que foi utilizado de terceiros.

Agora clique com o botão direito na pasta `common`, e vamos criar as pastas `css`, `images` e `js` via a opção `New → Folder`. Elas vão armazenar como os nomes indicam arquivos CSS, imagens e JavaScript gerados por nós.

Na pasta `lib`, por sua vez, clique com o botão direito sobre ela e crie as pastas `bootstrap` e `jQuery` via a opção `New → Folder`. Essa pasta irá armazenar as bibliotecas de terceiros que serão

utilizadas na aplicação.

Na pasta `bootstrap`, por sua vez, clique com o botão direito sobre ela e crie as pastas `css`, `fonts`, `images` e `js` via a opção `New → Folder`.

Já na pasta `jQuery`, por sua vez, clique com o botão direito sobre ela e crie a pasta `js` via a opção `New → Folder`.

A estrutura final do projeto ficará como mostrado na figura 3.26, com as pastas criadas e os arquivos PHP desenvolvidos. Ainda iremos popular as pastas criadas com os arquivos acessórios nossos e de terceiros também.

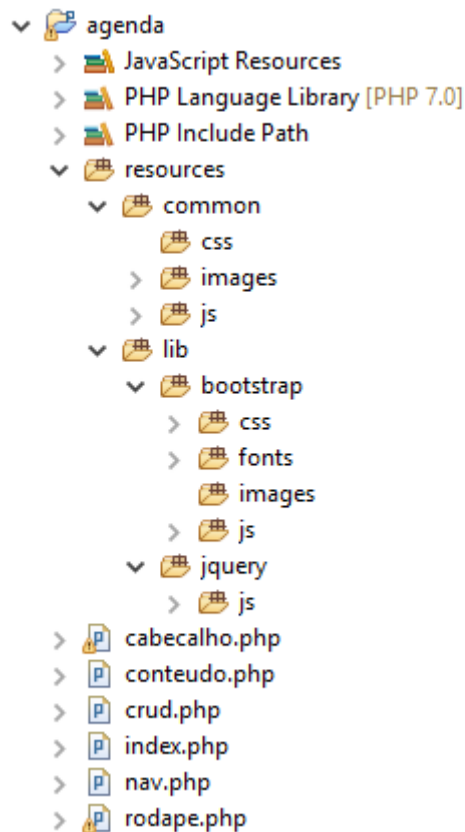


Figura 3.26 —Estrutura final do projeto agenda.

3.18.3 Baixando a biblioteca Bootstrap

O projeto fará uso da biblioteca Bootstrap, para obtê-la acesse o site <http://getbootstrap.com/getting-started/#download>. A opção escolhida foi clicar no botão `Download Bootstrap` para baixar a última versão disponível quando da publicação desta obra. A figura 3.27 mostra isso.

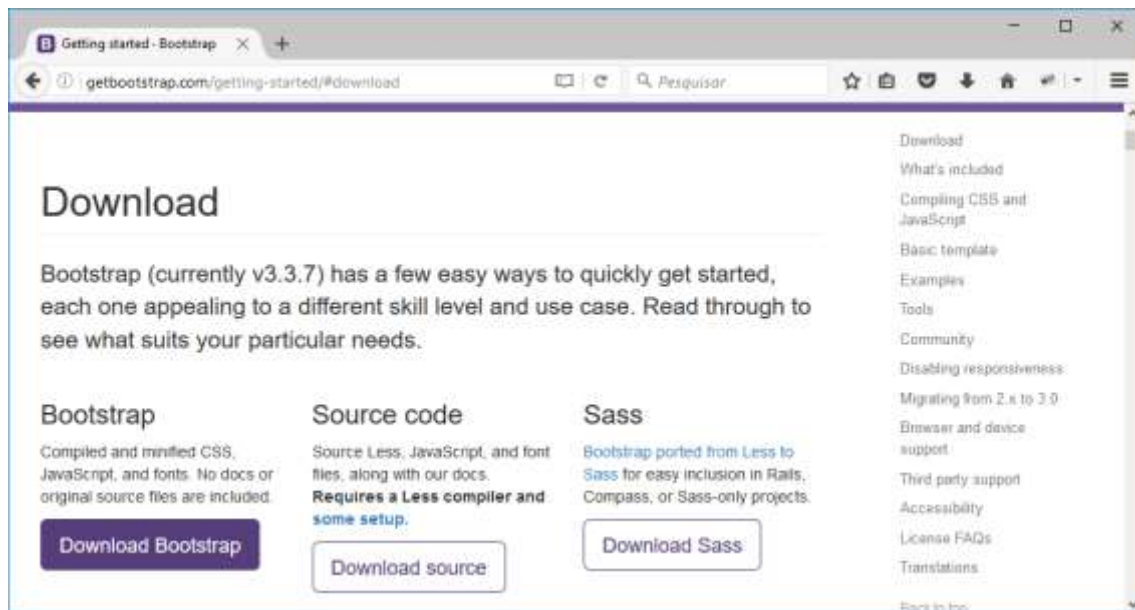


Figura 3.27 —Download da biblioteca Bootstrap.

O arquivo disponibilizado para download foi o `bootstrap-3.3.7-dist.zip`, descompacte o seu conteúdo em uma pasta de sua preferência. A estrutura dessa versão mostra algo como listado na figura 3.28.

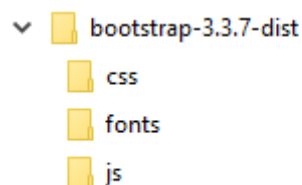


Figura 3.28 —Conteúdo do diretório da biblioteca Bootstrap.

Copie essas pastas baixadas para dentro do projeto agenda no Eclipse. A estrutura dentro da IDE com os respectivos arquivos que serão utilizados no projeto fica como mostrando na figura 3.29.

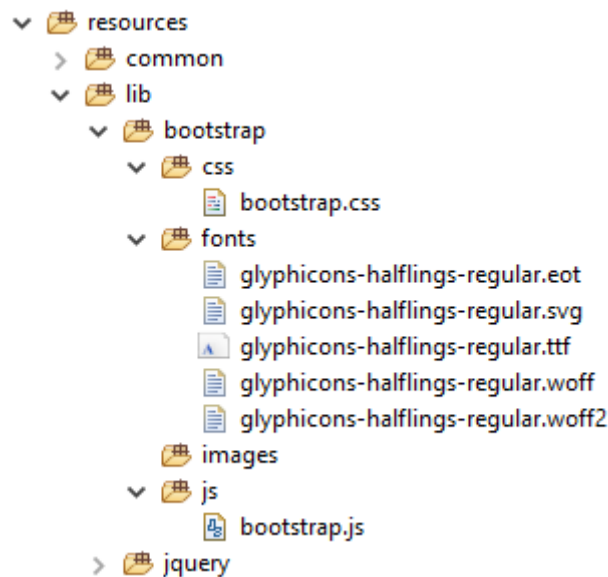


Figura 3.29 —Bootstrap no projeto agenda.

No caso da pasta `css`, deixamos somente o arquivo `bootstrap.css`. E na pasta `js` deixamos somente o arquivo `bootstrap.js`. Uma vez que não iremos utilizar os outros arquivos dessas fornecidos pela biblioteca Bootstrap.

3.18.4 Baixando a biblioteca jQuery

O projeto também fará uso da biblioteca jQuery, para obtê-la acesse o site <http://jquery.com/download/>. A opção escolhida foi clicar no link Download the uncompressed, development jQuery 3.1.1, para baixar a última versão disponível quando da publicação desta obra. A figura 3.30 mostra isso.

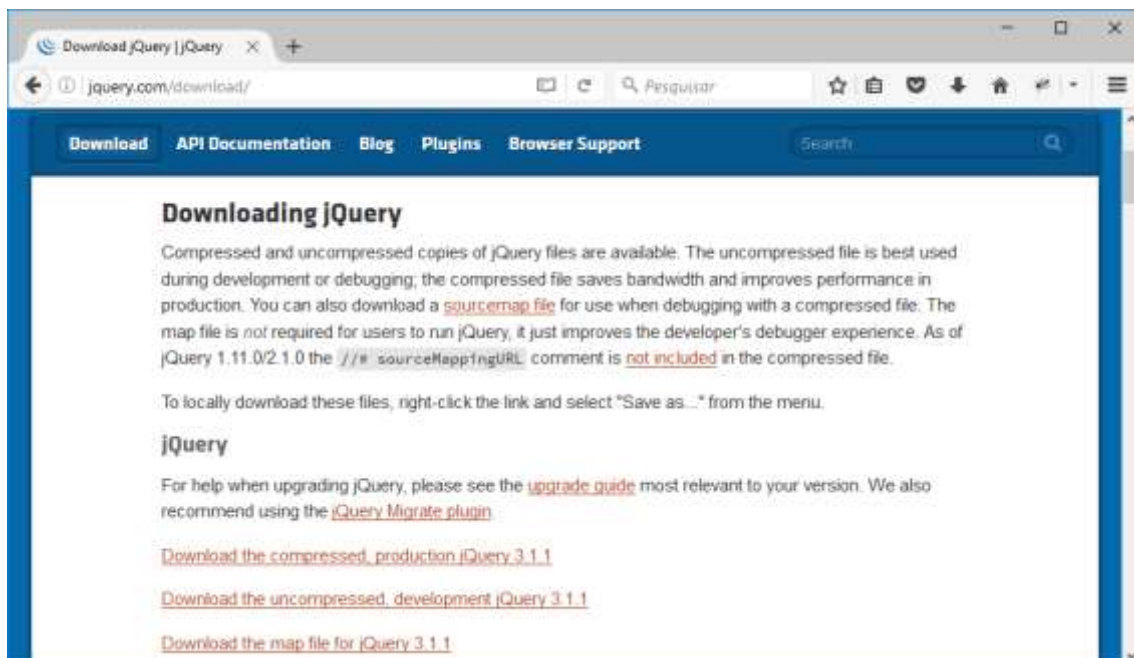


Figura 3.30 —Download da biblioteca jQuery.

O arquivo disponibilizado para download foi o `jquery-3.1.1.js`, esse arquivo pode ser copiado diretamente para `jquery/js`, dentro do projeto agenda do Eclipse. A figura 3.31 mostra isso.

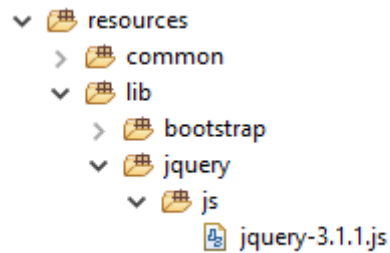


Figura 3.31 —jQuery no projeto agenda.

Com isso terminamos de copiar as bibliotecas de terceiros necessárias para o nosso projeto.

3.18.5 Programando o projeto

Agora vamos criar os códigos-fontes necessários para dar continuidade ao projeto. O projeto será separado em vários arquivos fontes visando otimizar a manutenção do mesmo, e também para separar responsabilidades. A figura 3.32 demonstra como irá ser o fluxo dos dados na aplicação.

Projeto agenda

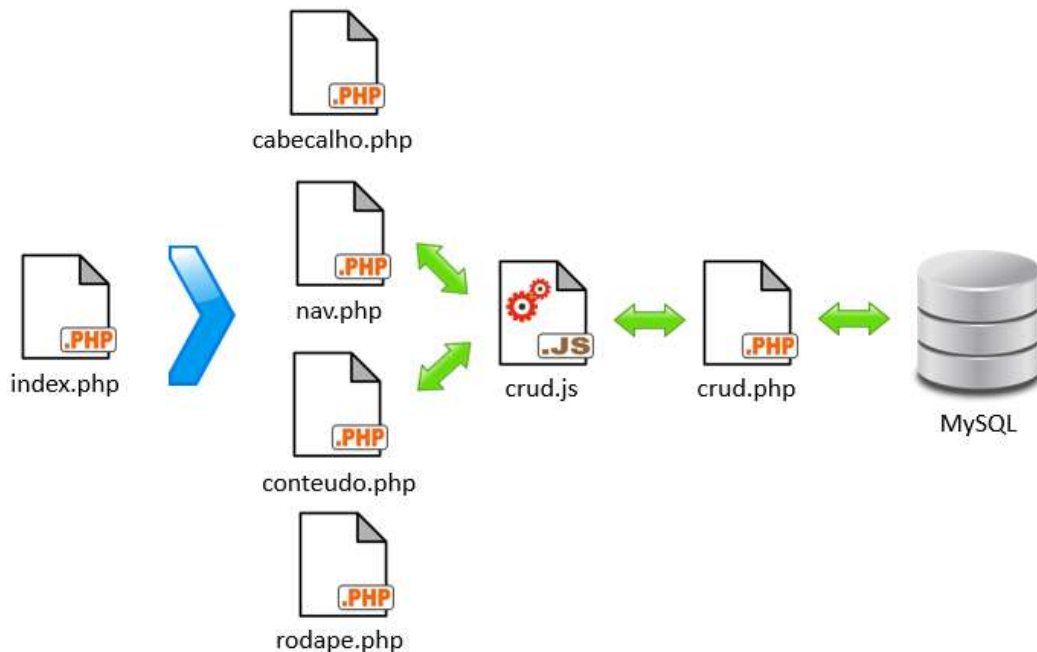


Figura 3.32 —Fluxo das informações no projeto agenda.

3.18.5.1 Criando o index.php

Esse arquivo é o principal do projeto, apesar de não conter código puro, ele será o responsável por chamar os códigos-fontes PHP responsáveis por criar o nosso cadastro. Para construir esse código-fonte clique com o botão direito sobre o nome do projeto *agenda*, vá então na opção New → PHP File e crie o arquivo *index.php* com o conteúdo a seguir. Repita esses passos sempre que for criar um novo arquivo php para o projeto.

index.php

```
<?php include("cabecalho.php"); ?>
```

```
<?php include("nav.php"); ?>❷  
<?php include("conteudo.php"); ?>❸  
<?php include("rodape.php"); ?>❹
```

Em ^❶ fazemos a importação da página PHP que terá as informações do cabeçalho do sistema.

Em ^❷ fazemos a importação da página PHP que será uma barra de ferramentas que contém a parte de listagem e de cadastro novo do sistema.

No ponto ^❸ fazemos a importação da página PHP que irá receber a listagem dos dados e outras operações como edição e exclusão.

Em ^❹ estará o código que fará as importações JavaScript visando diminuir o tempo de carregamento da página.

3.18.5.2 Criando o cabeçalho.php

Vamos criar o próximo arquivo PHP do projeto o `cabeçalho.php`. Esse novo fonte contém as informações de cabeçalho da página e a importação do CSS do Bootstrap, além das aplicações dos estilos principais que vamos utilizar no projeto. Esse código-fonte só contém instruções HTML.

 `cabeçalho.php`

```
<!DOCTYPE html>  
<html>  
<head>  
<meta charset="UTF-8">  
<meta http-equiv="X-UA-Compatible" content="IE=edge">❶  
<meta name="viewport" content="width=device-width, initial-scale=1">❷  
<title>CRUD Contato - PHP Profissional</title>  
<link href="./resources/lib/bootstrap/css/bootstrap.css"  
  rel="stylesheet">❸  
<body>  
  <div class="container">❹  
    <div class="jumbotron">❺  
      <h1>CRUD - Contato - PHP Profissional</h1>  
    </div>
```

Em ^❶, usamos uma `meta` tag que diga ao Internet Explorer, que ele tente mostrar a página da maneira mais compatível com os padrões web que ele consiga. Outros navegadores ignoram essa diretiva.

Em ^❷ essa tag auxilia aos navegadores de dispositivos móveis a mostrar a página de maneira mais efetiva. Essa instrução diz ao navegador, para colocar a largura da página do mesmo tamanho da largura da tela do dispositivo e com um zoom de 100%. Essa instrução facilita a compatibilidade com vários dispositivos com resoluções diferentes.

No ponto ^❸ foi feito a importação da biblioteca Bootstrap.

Em ^❹ estamos usando um dos primeiros estilo do Bootstrap o `container`, repare que a `div` que contém esse estilo não será fechada aqui somente no arquivo `rodape.php`, quando finalizarmos o

layout do projeto. Essa classe `container` serve para usarmos em layouts com tamanhos fixos. Além disso essa classe de estilo, é uma das bases do sistema de grid do Bootstrap. A outra é a classe `container-fluid` que não trabalha com layouts fixos, usando toda a tela disponível. Apesar de não termos um layout fixo no projeto optou-se pelo uso da classe `container`.

Em ❶ utilizou-se a classe `jumbotron`, esse estilo irá destacar mensagem do nosso site.

3.18.5.3 Criando o `nav.php`

O fonte `nav.php`, contém as informações das duas ações que vamos realizar sob o nosso cadastro que são listar e chamar a inclusão de um novo contato. Além de ter uma `div` escondida que é chamada sempre como um indicador de carregamento da página. Esse código-fonte também só contém instruções HTML.

 `nav.php`

```
<nav class="navbar navbar-default">❶
  <div class="container">❷
    <ul class="nav navbar-nav">
      <li><a href="javascript:void(0);" id="listar_contato"><i
        class="glyphicon glyphicon-th-list"></i> Listar</a></li>❸
      <li><a href="javascript:void(0);" id="novo_contato"><i
        class="glyphicon glyphicon-plus"></i> Novo</a></li>❹
    </ul>
  </div>
</nav>
<div id="carregando" style="display: none; text-align: center;">❺
  
</div>
```

Em ❶, usamos o estilo `navbar` padrão para identificar o estilo que irá armazenar botões do projeto.

Em ❷, aplicamos a classe `container` que é responsável pelo sistema de grid do bootstrap.

No ponto ❸, listamos o nosso primeiro link “botão” que será responsável por chamar um evento javascript que responderá pela listagem de contatos. Também criamos um identificador (`id`) para esse link, a atribuição dessa propriedade é importante pois precisamos dessa informação adiante. Repare também que usamos uma função javascript que não retorne nada dentro da propriedade `href`, isso é foi necessário porque iremos injetar conteúdo HTML no arquivo `conteudo.php` quando esse link for clicado.

No ponto ❹, listamos o nosso segundo link “botão” que será responsável por chamar um evento javascript que responderá pela visualização do formulário de inclusão. Também criamos um identificador (`id`) para esse link, a atribuição dessa propriedade é importante pois também iremos precisar dessa informação adiante. Repare que novamente também usamos uma função javascript que não retorne nada dentro da propriedade `href`, isso é foi necessário porque iremos injetar novamente conteúdo HTML no arquivo `conteudo.php` quando esse link for clicado.

Por fim em ❺, criamos uma `div` que fica escondida pois usamos uma instrução para tal via a propriedade `style`, aonde foi especificado que ela não deve ser mostrada `display : none`, mas quando o for, o seu conteúdo se mostrará de forma centralizada `text-align : center`. Essa `div` irá

mostrar uma imagem de carregamento sempre que for solicitada que apareça. A imagem criada pode ser obtida e customizada a gosto do leitor através do site <http://ajaxload.info/>, no livro usamos a opção Wheel Throbber. Essa imagem fica no local especificado para imagens do nosso projeto conforme o caminho apontado ali na propriedade `src`.

3.18.5.4 Criando o `conteudo.php`

O fonte `conteudo.php`, é o fonte responsável por mostrar os dados do projeto. Nele encontraremos duas `divs`, uma é onde será injetado o conteúdo com dados do projeto. A outra é um diálogo que mostra se um contato será excluído ou não do banco. Novamente vemos a presença de somente instruções HTML no fonte.

 `conteudo.php`

```
<!-- exibição de conteúdo -->
<div id="conteudo"></div>❶
<!-- caixa de exclusão -->
<div id="excluir_contato_modal" class="modal fade" tabindex="-1"
  role="dialog">❷
  <div class="modal-dialog" role="document">
    <div class="modal-content">❸
      <div class="modal-header">
        <button type="button" class="close" data-dismiss="modal"
          aria-label="Fechar">
          <span aria-hidden="true">&times;</span>
        </button>
        <h4 class="modal-title">Excluir Contato</h4>
      </div>
      <div class="modal-body">
        <p>Confirma a exclusão do contato?</p>
      </div>
      <div class="modal-footer">❹
        <input type="hidden" id="contato_id" value="" />
        <button type="button" id="excluir" class="btn btn-primary">Sim</button>
        <button type="button" class="btn btn-default" data-dismiss="modal">Não</button>
      </div>
    </div>
  </div>
</div>
```

Em ❶, temos a `div` identificada como `conteudo`, ela que irá receber os dados vindos de operações como listar, editar e incluir via injeção de HTML nessa `div`.

Em ❷, construímos uma `div` que tem um diálogo Bootstrap responsável pela confirmação da exclusão de um registro, diálogo esse que só irá aparecer quando acionado.

No ponto ❸, começamos a construir o corpo do diálogo que contém as mensagens que o usuário irá ver. Esse diálogo está dividido em cabeçalho, corpo e rodapé. Representando respectivamente pelas classes `modal-header`, `modal-body` e `modal-footer`.

No ponto ❹, listamos os botões desse diálogo a opção `Sim` dispara uma ação JavaScript já a opção `Não` fecha o diálogo. A opção que confirma a exclusão é primária desse diálogo isso é confirmado via classe `btn btn-primary`.

3.18.5.5 Criando o rodape.php

O fonte `rodape.php`, é o mais simples do projeto. Ele contém o fechamento da `div` aberta e não fechada no arquivo `cabecalho.php`. E as importações do JavaScripts necessários ao projeto. Aqui também temos novamente só instruções HTML.

 `rodape.php`

```
</div>❶
<script type="text/javascript"
  src="./resources/lib/jquery/js/jquery-3.1.1.js"></script>❷
<script type="text/javascript"
  src="./resources/lib/bootstrap/js/bootstrap.js"></script>
<script type="text/javascript" src="./resources/common/js/crud.js"></script>
</body>
</html>
```

Em ^❶, fechamos a `div` iniciada no `cabecalho.php`, delimitando assim layout do nosso projeto.

Em ^❷, realizamos a importação de vários arquivos javascripts necessários ao nosso projeto. Sendo que a última importação é um arquivo que ainda iremos criar o `crud.js`. Os outros arquivos já estão disponíveis pois copiamos os mesmos quando da criação do projeto. Importar arquivos javascripts somente no final é uma boa prática que visa otimizar o carregamento da página.

3.18.5.6 Criando o crud.js

Esse arquivo é o motor da nossa aplicação tudo o que fizermos passará por ele, fazendo a ponte do lado cliente com o lado servidor. Esse arquivo será mostrado por partes visando melhor compreender o seu funcionamento. Inicialmente vamos criar os cabeçalhos das funções que iremos utilizar, lembre de salvar o arquivo na pasta `js` da `common`.

 `crud.js`

```
$(function() {
  $(document).on("click", "a#listar_contato", function() {❶
    getListarContato();
  });
  $(document).on("click", "a#novo_contato", function() {❷
    getFormularioInclusao();
  });
  $(document).on("click", "button#salvar_contato", function() {❸
    if (validaDados()) {
      salvarContato();
    }
  });
  $(document).on("click", "a#confirma_excluir", function() {❹
    confirmaExclusao(this);
  });
  $(document).on("click", "button#excluir", function() {❺
    excluirContato();
  });
});
```

```

$(document).on("click", "a#editar_contato", function() {❸
    getEditarContato(this);
});
$(document).on("click", "button#salvar_contato_edicao", function() {❹
    if (validaDados()) {
        salvarContatoEdicao();
    }
});
$(document).on("focus", "input#nome", function() {❺
    $("input").removeClass('error').next('span').remove();
});
});

```

Antes do ponto ❶, fizemos uso da instrução `$(function() {` do jQuery, que é finalizada na última parte do código com a instrução `});`. Essa instrução é equivalente a `$(document).ready(function() {});`, que indica que o código só estará disponível para execução após o carregamento do modelo DOM da página. Depois já no ponto ❶, modelamos um evento de clique que será disparado ao quando o link `listar_contato`, for acionado, esse link foi criado no fonte `nav.php`. Fizemos isso via instrução `$(document).on("click", "a#listar_contato", function() {`. Essa instrução indica ao jQuery que queremos adicionar um evento a um documento (`index.php`), esse evento será do tipo clique, e será disparado via um link que identificamos como `listar_contato`, e por fim modelamos função que irá responder ao evento. Dentro da função temos a chamada à função `getListarContato();` que será responsável por listar os dados que precisamos.

Em ❷, a exemplo da explicação do ponto ❶ temos a chamada de função que será responsável por montar o nosso formulário de inclusão `getFormularioInclusao();`.

Em ❸, temos uma chamada que é responsável por disparar duas funções `validaDados` e `salvarContato`. A primeira é uma verificação para ver se o campo `nome` do contato será preenchido, uma vez que marcamos ele como obrigatório quando da criação da nossa tabela, essa função irá retornar verdadeiro no caso de tudo estar correto. E a outra irá disparar a rotina para salvar o contato.

No ponto ❹, tem a chamada de função `confirmaExclusao(this);` que recebe como parâmetro o link que foi criado para chamar o diálogo de exclusão. Esse link contém também o código do contato que iremos excluir.

O ponto ❺, tem a chamada de função `excluirContato` que irá excluir o contato que queremos.

O ponto ❻, tem a chamada de função `getEditarContato(this);` que irá buscar os dados do contato que iremos editar, ela recebe como parâmetro o link que contém o contato que queremos editar.

O ponto ❼, tem uma validação de dados e salva as informações editadas, a exemplo do que aconteceu no ponto ❸.

Por fim o ponto ❽, remove o aviso para preenchimento do campo `nome` caso ele não tenha sido preenchido. Essa remoção ocorre quando o campo recebe o foco.

Essa é a estrutura principal do arquivo `crud.js`, continuaremos a editar o mesmo detalhando agora o corpo das funções. Após esse trecho de código inicial, basta ir adicionando o corpo das funções conforme a seguir. Entretanto já é possível ver como o nosso projeto irá ficar, basta acessar o endereço do seu projeto em <http://localhost/agenda>, a figura 3.33 mostra isso.

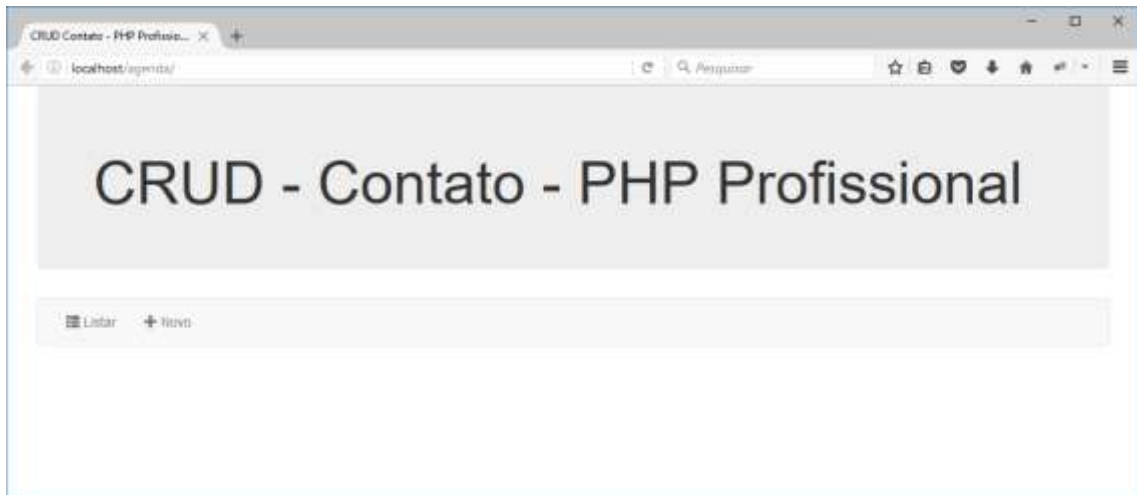


Figura 3.33 —Imagem do projeto agenda em execução.

Checklist

Na ocorrência de algum problema na execução do teste, confira o que construímos até aqui no projeto para garantir que você não se esqueceu de nenhum detalhe.

- ☒ Criar o PHP Project agenda;
- ☒ Adicionar as bibliotecas de terceiros (Seções 3.19.3 e 3.19.4);
- ☒ Verificar se o banco de dados agenda foi criado via MySQL 5.7 Command Line Client;
- ☒ Executar o comando `USE agenda`, para tornar o banco de dados agenda ativo para os próximos comandos;
- ☒ Criar a tabela contato (Seção 3.19.1);
- ☒ Verificar se o servidor apache está no ar para acessar a URL de teste <http://localhost/agenda>.

Detalhando `getListarContato()`

Essa função irá consultar os dados no lado servidor. Adicione isso ao código já existente do `crud.js`.

 `crud.js`

```
[...]
function getListarContato() {
    $('#carregando').show(); ❶

    $.post('crud.php', { ❷
        acao : 'listar_contato'
    }, function(dados) {
        criaListagemTabela(dados); ❸
        $('#carregando').hide();
    }, "json");
}
```

Aqui em ❶, realizamos o comando `$('#carregando').show();` que é a div que irá apresentar uma animação indicando o carregamento dos dados, mostrando assim ao usuário que a aplicação não está “congelada” mas em processamento. Essa div foi criada na página `nav.php`.

Em ❷, temos o comando jQuery `$.post`, essa função é muito parecido com `$.ajax` dessa biblioteca, com a diferença que só trabalha com o envio de dados via POST. Essa função espera alguns parâmetros o primeiro é a URL que irá receber a solicitação no caso `crud.php`, o segundo são os dados que se quer enviar para o servidor no caso `{acao : 'listar_contato'}`, o terceiro parâmetro uma função para lidar com o retorno do processamento que virá na variável `dados`, e finalmente o quarto parâmetro é o formato de retorno que se espera no processamento do lado servidor aqui optou-se pelo formato `json`.

Em ❸, é trabalhado o retorno da consulta ao fonte `crud.php`, de forma que a função lide com a variável `dados` e seu conteúdo, processando a mesma em outra função `criaListagemTabela(dados)`. Ao final do processamento a `div` que mostra animação de carregamento é novamente escondida via comando `$('#carregando').hide()`.

Detalhando `criaListagemTabela()`

Essa função irá montar a tabela de visualização dos dados dos contatos cadastrados.

 `crud.js`

```
[...]
function criaListagemTabela(jsonDados) {❶
    var tabela = '<div class="table-responsive">';❷
    tabela += '<table class="table table-hover table-bordered"><thead><tr>';❸
    tabela += '<th scope="col">#</th><th scope="col">Nome</th>'
    tabela += '<th scope="col">Telefone</th><th scope="col">E-mail</th>';
    tabela += '<th scope="col">Endereço</th><th scope="col"></th><th scope="col"></th></tr></thead><tbody>';

    $.each(
        jsonDados,
        function(indice, contato) {❹
            tabela += '<tr>';
            tabela += '<td>' + contato.id + '</td>';
            tabela += '<td>' + contato.nome + '</td>';
            tabela += '<td>' + contato.telefone + '</td>';
            tabela += '<td>' + contato.email + '</td>';
            tabela += '<td>' + contato.endereco + '</td>';
            tabela += '<td><a href="javascript:void(0);" id="editar_contato" contato_id="'
                + contato.id
                + '" class="btn btn-success btn-xs"><i class="glyphicon glyphicon-pencil"></i></a></td>';❺
            tabela += '<td><a href="javascript:void(0);" id="confirma_excluir" contato_id="'
                + contato.id
                + '" class="btn btn-danger btn-xs"><i class="glyphicon glyphicon-trash"></i></a></td>';❻
            tabela += '</tr>';
        });

    tabela += '</tbody></table></div>';
    $('#div#conteudo').html(tabela);❼
}
```

Aqui em ❶, temos a função `function criaListagemTabela(jsonDados) {`, recebendo os dados vindo

do servidor via variável `jsonDados`.

Em ❷, criamos uma variável denominada `tabela` que vai armazenar os comandos HTML, de forma a termos uma `div` que por sua vez conterá uma tabela que exibirá os dados consultados. A classe aplicada a `div`, se chama `class="table-responsive"` e ela irá apresentar uma barra de rolagem com dispositivos abaixo de 768 pixels.

Em ❸, nesse ponto colocamos a classe `class="table table-hover table-bordered"` como a classe padrão da tabela, essa classe irá atribuir uma borda a tabela e destacar as linhas de dados quando o cursor estiver por cima.

Em ❹, a função `$.each` da biblioteca jQuery irá percorrer os dados estilo uma estrutura de repetição `foreach` do PHP. Essa função espera os dados que serão percorridos, e a função que irá lidar com eles. Repare que cada célula da tabela recebe um dado de informação do contato.

Em ❺, temos um botão que será responsável por disparar a edição do registro, e que recebeu a devida identificação.

Em ❻, temos um botão que será responsável por disparar a edição do registro, e que recebeu a devida identificação além do código do registro que será editado.

Em ❼, fazemos a injeção do conteúdo HTML na `div` denominada `conteudo`. Que foi criado no fonte `conteudo.php`.

Detalhando `getFormularioInclusao()`

Essa função irá montar o formulário de inclusão dos dados do contato.

 `crud.js`

[...]

```
function getFormularioInclusao() {❶
    var form = '<form class="form-horizontal">';❷
    form += '<div class="form-group">';❸
    form += '<label for="nome" class="col-sm-2 control-label glyphicon glyphicon-user"></label>';
    form += '<div class="col-sm-8">';
    form += '<input type="text" class="form-control" id="nome" placeholder="Nome*" required>';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<label for="telefone" class="col-sm-2 control-label glyphicon glyphicon-phone"></label>';
    form += '<div class="col-sm-8">';
    form += '<input type="tel" class="form-control" id="telefone" placeholder="Telefone">';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<label for="email" class="col-sm-2 control-label glyphicon glyphicon-envelope"></label>';
    form += '<div class="col-sm-8">';
    form += '<input type="email" class="form-control" id="email" placeholder="E-mail">';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<label for="endereco" class="col-sm-2 control-label glyphicon glyphicon-home"></label>';
```

```

form += '<div class="col-sm-8">';
form += '<textarea class="form-control" rows="3" id="endereco" placeholder="Endereço"></textarea>';
form += '</div>';
form += '</div>';
form += '<div class="form-group">';
form += '<div class="col-sm-offset-2 col-sm-10">';
form += '<button type="button" id="salvar_contato" class="btn btn-primary">Salvar</button>';
form += '</div>';
form += '</div>';
form += '</form>';

$('div#conteudo').html(form); ❹
}

```

Aqui em ❶, temos a função `function criaListagemTabela(jsonDados) {`, recebendo os dados vindo do servidor via variável `jsonDados`.

Em ❷, temos a criação de uma variável denominada `form`, essa variável é responsável por armazenar todo o conteúdo de criação de um formulário HTML. Nesse mesmo ponto, estamos definindo qual será a classe Bootstrap para formatar visualmente o formulário optou-se pela classe `class="form-horizontal"`, de forma que os elementos fiquem organizados de forma horizontal. Com um identificador (mensagem) ao lado do campo.

Em ❸, é definido um `div` para cada conjunto identificador e campo. Para agrupar essa estrutura usou-se a classe do Bootstrap `class="form-group"`. Cada componente dessa classe seja a mensagem ao lado do campo ou campo em si, ocupa um tamanho dentro do sistema de grid do bootstrap que comporta até 12 colunas. Optou-se por 2 para o identificador (mensagem) e 8 para o campo em si.

Em ❹, fazemos a injeção do conteúdo HTML na `div` denominada `conteudo`. Que foi criado no fonte `conteudo.php`.

Detalhando salvarContato()

Essa função irá chamar a rotina para salvar o contato no lado servidor.

 crud.js

```

[...]
```

```

function salvarContato() {
    $('#carregando').show(); ❶

    var Contato = new Object(); ❷
    Contato.nome = $('#input#nome').val();
    Contato.telefone = $('#input#telefone').val();
    Contato.email = $('#input#email').val();
    Contato.endereco = $('#textarea#endereco').val();

    var contatoJson = JSON.stringify(Contato); ❸

    $.post('crud.php', { ❹
        acao : 'adicionar_contato',

```



```

        contato : contatoJson
    }, function(dado) {
        getListarContato();
        $('#carregando').hide();
    }, "json");
}

```

Aqui em ❶, realizamos o comando `$('#carregando').show();` que é a `div` que irá apresentar uma animação indicando o carregamento dos dados, mostrando assim ao usuário que a aplicação não está “congelada” mas em processamento. Essa `div` foi criada na página `nav.php`.

Em ❷, temos a criação de uma classe JavaScript para representar o contato, repare que ele recebe como propriedade cada um dos valores digitados nos campos do formulário. As propriedades dessa classe foram criadas conforme a quantidade de campos que tínhamos na tabela `contato` e que gostaríamos de salvar. Só não foi utilizado o `id` pois é um campo auto incremento no banco de dados.

Em ❸, foi utilizada a função `JSON.stringify` que converte valores do JavaScript em uma string JSON. Assim o objeto que criamos irá gerar uma string tipo `{nome: fulano, telefone: 123 ...}` para ser processada no servidor.

Em ❹, temos o comando jQuery `$.post`, essa função é muito parecido com `$.ajax` dessa biblioteca, com a diferença que só trabalha com o envio de dados via POST. Essa função espera alguns parâmetros o primeiro e a URL que irá receber a solicitação no caso `crud.php`, o segundo são os dados que se quer enviar para o servidor no caso `{acao : 'adicionar_contato', contato : contatoJson}`, o terceiro parâmetro uma função para lidar com o retorno do processamento que virá na variável `dados`, e finalmente o quarto parâmetro é o formato de retorno que se espera no processamento do lado servidor aqui optou-se pelo formato `json`.

Detalhando `confirmaExclusao()` e `excluirContato()`

Essas funções disparam o diálogo de exclusão e em o botão de confirmação desse diálogo.

 `crud.js`

```

[...]
function confirmaExclusao(elemento) {❶
    $("#excluir_contato_modal").modal("show");
    $("#excluir_contato_modal input#contato_id").val(❷
        $(elemento).attr('contato_id'));
    }

function excluirContato() {
    var Contato = new Object();❸
    Contato.id = $("#excluir_contato_modal input#contato_id").val();

    var contatoJson = JSON.stringify(Contato);

    $.post('crud.php', {❹
        acao : 'excluir_contato',
        contato : contatoJson
    }, function(dado) {
        getListarContato();
    }

```

```

        $("#excluir_contato_modal").modal("hide");
    }, "json");
}

```

Aqui em ❶, a função `confirmaExclusao` recebe como parâmetro (variável `elemento`) o link de exclusão que foi clicado e que contém o `id` do contato que se quer excluir.

Em ❷, recuperamos o `id` do contato que se quer excluir e passamos o mesmo para o diálogo de exclusão.

Em ❸, temos a criação de uma classe JavaScript para representar o contato, repare que aqui foi utilizado um único atributo o `id`, pois esse campo é chave primária da tabela e que será usado para instrução SQL para excluir o contato.

Em ❹, temos o comando jQuery `$.post`, essa função é muito parecido com `$.ajax` dessa biblioteca, com a diferença que só trabalha com o envio de dados via POST. Essa função espera alguns parâmetros o primeiro e a URL que irá receber a solicitação no caso `crud.php`, o segundo são os dados que se quer enviar para o servidor no caso `{acao : 'excluir_contato', contato : contatoJson}`, o terceiro parâmetro uma função para lidar com o retorno do processamento que virá na variável `dados`, e finalmente o quarto parâmetro é o formato de retorno que se espera no processamento do lado servidor aqui optou-se pelo formato `json`. Essa etapa do código esconde o diálogo de exclusão e lista os dados novamente caso tenha-se optado por tal operação confirmando a exclusão.

Detalhando `getEditarContato()`, `getFormularioEdicao()` e `salvarContatoEdicao()`

Essas funções realizam respectivamente a busca de um contato para edição, a montagem de um formulário com dados para edição e o salvamento dos dados editados.

 `crud.js`

```

[...]
```

```

function getEditarContato(elemento) {❶
    var Contato = new Object();
    Contato.id = $(elemento).attr('contato_id');

    var contatoJson = JSON.stringify(Contato);

    $('#carregando').show();

    $.post('crud.php', {
        acao : 'buscar_contato',
        contato : contatoJson
    }, function(dado) {
        getFormularioEdicao(dado);
        $('#carregando').hide();
    }, "json");
}

function getFormularioEdicao(jsonDado) {❷
    var form = '<form class="form-horizontal">';
    $
        .each(

```

```

jsonDado,
function(indice, contato) {
    form += '<div class="form-group">';
    form += '<input type="hidden" id="contato_id" value="'
        + contato.id + '">';
    form += '<label for="nome" class="col-sm-2 control-label glyphicon glyphicon-user"></label>';
    form += '<div class="col-sm-8">';
    form += '<input type="text" class="form-control" id="nome" placeholder="Nome*" value="'
        + contato.nome + '" required>';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<label for="telefone" class="col-sm-2 control-label glyphicon glyphicon-';
phone"></label>';
    form += '<div class="col-sm-8">';
    form += '<input type="tel" class="form-control" id="telefone" placeholder="Telefone" value="'
        + contato.telefone + '">';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<label for="email" class="col-sm-2 control-label glyphicon glyphicon-';
envelope"></label>';
    form += '<div class="col-sm-8">';
    form += '<input type="email" class="form-control" id="email" placeholder="E-mail" value="'
        + contato.email + '">';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<label for="endereco" class="col-sm-2 control-label glyphicon glyphicon-home"></label>';
    form += '<div class="col-sm-8">';
    form += '<textarea class="form-control" rows="3" id="endereco" placeholder="Endereço">'
        + contato.endereco + '</textarea>';
    form += '</div>';
    form += '</div>';
    form += '<div class="form-group">';
    form += '<div class="col-sm-offset-2 col-sm-10">';
    form += '<button type="button" id="salvar_contato_edicao" class="btn btn-';
primary">Salvar</button>';
    form += '</div>';
    form += '</div>';
    });
    form += '</form>';
    $('div#conteudo').html(form);
}

function salvarContatoEdicao() {
    $('#carregando').show();

    var Contato = new Object();
    Contato.id = $('input#contato_id').val();

```

```

Contato.nome = $('#input#nome').val();
Contato.telefone = $('#input#telefone').val();
Contato.email = $('#input#email').val();
Contato.endereco = $('#textarea#endereco').val();

var contatoJson = JSON.stringify(Contato);

$.post('crud.php', {
  acao : 'editar_contato',
  contato : contatoJson
}, function(dado) {
  getListaContato();
  $('#carregando').hide();
}, "json");
}

```

Aqui em ❶, a função `getEditarContato` recebe como parâmetro (variável elemento) o link de edição que foi clicado e que contém o `id` do contato que se quer editar. O contato então é recuperado para edição.

Em ❷, com o contato recuperado é montado um formulário igual a edição, com exceção que os campos virão com valores preenchidos.

Em ❸, temos o comando jQuery `$.post`, essa função é muito parecido com `$.ajax` dessa biblioteca, com a diferença que só trabalha com o envio de dados via POST. Essa função espera alguns parâmetros o primeiro e a URL que irá receber a solicitação no caso `crud.php`, o segundo são os dados que se quer enviar para o servidor no caso `{acao : 'editar_contato', contato : contatoJson}`, o terceiro parâmetro uma função para lidar com o retorno do processamento que virá na variável `dados`, e finalmente o quarto parâmetro é o formato de retorno que se espera no processamento do lado servidor aqui optou-se pelo formato `json`. Essa etapa do código esconde a `div` de carregamento e lista os dados novamente.

Detalhando validaDados()

A última função que vamos comentar no arquivo `crud.js` é a função de validação do campo `nome`.

`crud.js`

```

[...]
```

```

function validaDados() {❶
  if ($("#nome").val().trim().length == 0) {
    ($("#nome").addClass('error')).after(
      '<span class="error">Por favor, preencha esse campo.</span>');
    return false;
  }
  return true;
}

```

Aqui em ❶, a função `validaDados` verifica se o campo `nome` tem algum conteúdo, se não tiver cria um aviso no campo para preenchimento, retornando falso para validação. Se tudo estiver correto é retornado verdadeiro e o script segue o seu fluxo.

3.18.5.7 Criando o crud.php

Chegamos no último código-fonte a ser desenvolvido no projeto o `crud.php`, ele é o responsável por fazer a conexão com o banco de dados.

 crud.php

```
<?php
const SERVIDOR = "localhost:3306";❶
const BANCO = "agenda";
const USUARIO = "root";
const SENHA = "root";

if (!isset ( $_POST ['acao'] )) {❷
    print json_encode ( 0 );
    return;
}

switch ( $_POST ['acao'] ) {❸
    case 'buscar_contato' :
        $contato = new stdClass ();
        $contato = json_decode ( $_POST ['contato'] );
        try {
            $sql = "select * from contato where id = ?";
            $conexao = new PDO ( "mysql:host=" . SERVIDOR . ";dbname=" . BANCO, USUARIO, SENHA );
            $pre = $conexao->prepare ( $sql );
            $pre->execute ( array (
                $contato->id
            ) );
            print json_encode ( $pre->fetchAll ( PDO::FETCH_ASSOC ) );
        } catch ( PDOException $e ) {
            echo "Erro!: " . $e->getMessage () . "<br>";
            die ();
        } finally {
            $conexao = null;
        }
        break;
    case 'listar_contato' :
        try {
            $sql = "select * from contato";
            $conexao = new PDO ( "mysql:host=" . SERVIDOR . ";dbname=" . BANCO, USUARIO, SENHA );
            $pre = $conexao->prepare ( $sql );
            $pre->execute ();
            print json_encode ( $pre->fetchAll ( PDO::FETCH_ASSOC ) );
        } catch ( PDOException $e ) {
            echo "Erro!: " . $e->getMessage () . "<br>";
            die ();
        } finally {
            $conexao = null;
        }
}
```

```

break;
case 'adicionar_contato' :
    $contato = new stdClass ();
    $contato = json_decode ( $_POST ['contato'] );
    try {
        $sql = "insert into contato(nome, telefone, email, endereco) VALUES (?, ?, ?, ?) ";
        $conexao = new PDO ( "mysql:host=" . SERVIDOR . ";dbname=" . BANCO, USUARIO, SENHA );
        $pre = $conexao->prepare ( $sql );
        $pre->execute ( array (
            $contato->nome,
            $contato->telefone,
            $contato->email,
            $contato->endereco
        ) );
        print json_encode ( $conexao->lastInsertId () );
    } catch ( PDOException $e ) {
        echo "Erro!: " . $e->getMessage () . "<br>";
        die ();
    } finally {
        $conexao = null;
    }
    break;
case 'excluir_contato' :
    $contato = new stdClass ();
    $contato = json_decode ( $_POST ['contato'] );
    try {
        $sql = "delete from contato where id=? ";
        $conexao = new PDO ( "mysql:host=" . SERVIDOR . ";dbname=" . BANCO, USUARIO, SENHA );
        $pre = $conexao->prepare ( $sql );
        $pre->execute ( array (
            $contato->id
        ) );
        print json_encode ( 1 );
    } catch ( PDOException $e ) {
        echo "Erro!: " . $e->getMessage () . "<br>";
        die ();
    } finally {
        $conexao = null;
    }
    break;
case 'editar_contato' :
    $contato = new stdClass ();
    $contato = json_decode ( $_POST ['contato'] );
    try {
        $sql = "update contato set nome = ?, telefone = ?, email = ?, endereco = ? where id = ? ";
        $conexao = new PDO ( "mysql:host=" . SERVIDOR . ";dbname=" . BANCO, USUARIO, SENHA );
        $pre = $conexao->prepare ( $sql );
        $pre->execute ( array (
            $contato->nome,
            $contato->telefone,

```

```

        $contato->email,
        $contato->endereco,
        $contato->id
    ) );
    print json_encode ( 1 );
} catch ( PDOException $e ) {
    echo "Erro!: " . $e->getMessage () . "<br>";
    die ();
} finally {
    $conexao = null;
}
break;
}

exit ();
?>

```

No ponto ❶, temos a definição das configurações de acesso ao banco que podem variar conforme a sua instalação.

Em ❷, somente as opções que definimos em `crus.js`, são então validadas. Caso se acesse esse fonte e não se tenha uma ação atribuída o processamento é interrompido.

Em ❸, conforme a ação que foi requisitada no lado cliente é realizada alguma operação sob o banco de dados que segue o seguinte padrão que cria um objeto `$contato`, e decodifica a string json vindo do lado cliente para atributos equivalente. Depois cria-se uma variável `$sql` que contém a instrução base dos comandos que serão realizados no banco. Cria-se também uma conexão PDO com o banco de dados. E se a instrução sql tiver parâmetros ela é então prepara para execução. O retorno sempre será uma anotação json. Em caso de erro esse é capturado e impresso e ao final o objeto de conexão com o banco sempre é liberado.

Se tudo ocorreu bem as imagens a seguir mostram o resultado do sistema, na figura 3.34 a listagem de dados dos contatos, e a figura 3.35 o cadastro de um contato.

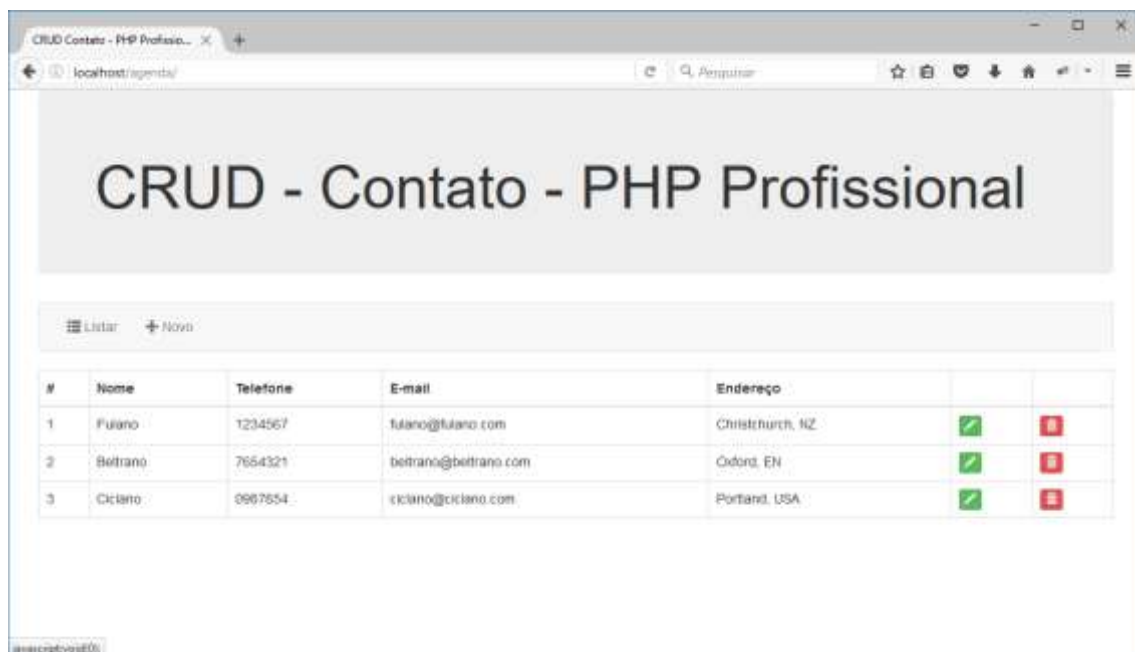


Figura 3.34 —Sistema agenda em funcionamento listagem de contatos.

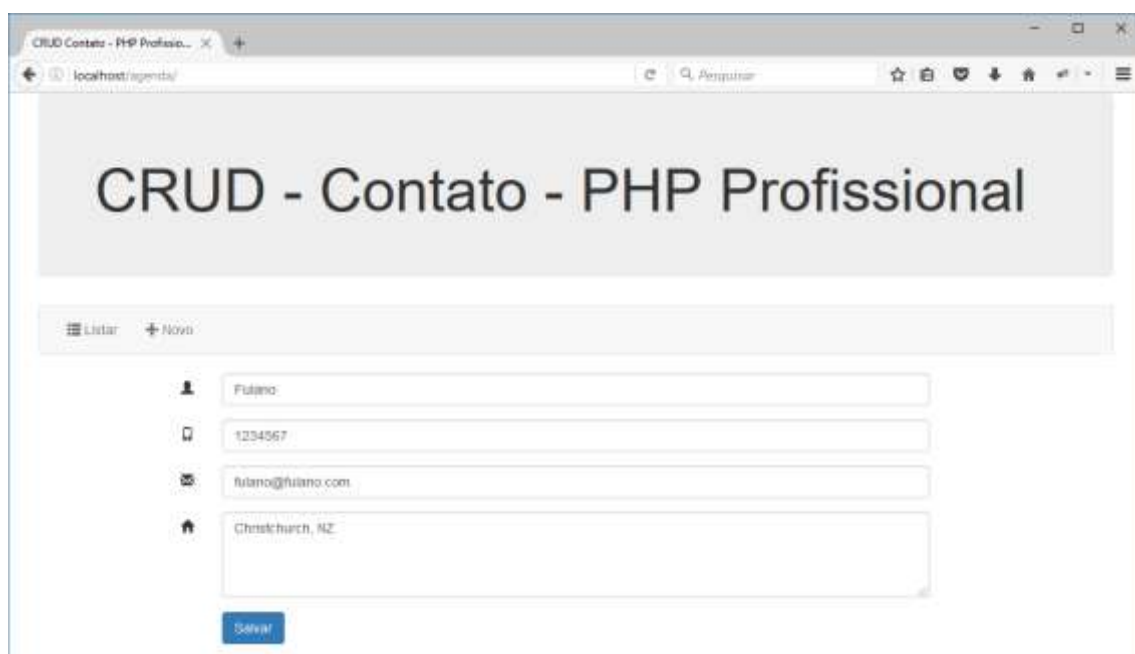


Figura 3.35 —Sistema agenda em funcionamento cadastro de contato.